

AI Prompts Used During Development

Complete Catalog of Actual Engineering Prompts Materially Employed Across Development Disciplines

Workflow Type	Categories Covered	Execution Platform	Total Prompts Cataloged
Step-by-Step Pair Programming	8 Engineering Disciplines	Antigravity Agentic IDE	24 Material Prompts

1. Overview & Methodology

The **AI Study Companion** was engineered through disciplined step-by-step pair programming with an advanced AI coding agent. Rather than attempting a single monolithic prompt, development proceeded sequentially across eight distinct engineering domains:

- Architecture & Tenant Isolation
- Frontend Engineering & React UI
- Backend & Document Ingestion
- Database Design & Batch Chunking
- AI & Multimodal OCR Engineering
- Debugging & Root-Cause Elimination
- Testing & Performance Benchmarking
- Documentation & Deployment

Below are the verbatim prompts materially utilized to scaffold, implement, refactor, and harden the codebase.

2. Architecture & Tenant Isolation Prompts

PROMPT 1.1 — MULTI-TENANT HIERARCHY DESIGN

Design a strict multi-tenant learning workspace in Node.js and MongoDB. The system must enforce a clean three-tier hierarchy: User → Space → Project → Materials, Conversations, and Quizzes. Ensure that learners can only view, edit, and query their own spaces and projects. Implement Express middleware (checkProjectAccess) that intercepts every project-scoped request and rejects unauthorized access with HTTP 403 Forbidden.

PROMPT 1.2 — ROLE-BASED ACCESS CONTROL (RBAC)

Implement a role-based access control architecture separating 'user' (Learners) from 'admin' (Instructors/Administrators). Learners must only access learning workspace routes (/projects, /materials, /tutor, /quizzes). Only accounts with role === 'admin' can access /api/admin observability routes. If a non-admin account attempts to log in to the admin console, reject with an explicit 403 Access Denied error.

PROMPT 1.3 — IDEMPOTENCY & DEBOUNCE GUARDS

Prevent duplicate space and project creation when a user double-clicks or rapidly presses the submit button. Implement a 15-second debounce window in both spaceController and projectController that detects identical title submissions from the same user within 15 seconds and safely returns the existing record instead of inserting a duplicate. On the frontend, disable submit buttons immediately upon click and display an animated spinner.

3. Frontend Engineering & React UI Prompts

PROMPT 2.1 — TABBED WORKSPACE LAYOUT

Build a responsive, modern dark-mode learning workspace using React 18, Vite, and Tailwind CSS v3. The layout must feature a clean tabbed project view: Materials (upload & indexing), AI Tutor (conversational RAG), Adaptive Quiz (CAT assessments), Concept Mastery (radar & progress bars), and Analytics (growth metrics). Include a floating citation preview, glowing mastery status badges, and smooth tab transitions.

PROMPT 2.2 — SERVER-SENT EVENTS (SSE) STREAMING CHAT

Implement real-time token streaming for the AI Tutor interface. Connect to the backend SSE endpoint (/api/tutor/stream/:projectId) using EventSource or fetch with a ReadableStream reader. As tokens arrive, append them smoothly to the active message bubble without re-rendering the entire chat history. Automatically parse out inline citation tags and render them as clickable pill buttons that jump to the cited document and page number.

PROMPT 2.3 — COMPUTER-ADAPTIVE QUIZ INTERFACE

Build an interactive 7-question adaptive quiz UI. The interface must display an Adaptive Trajectory Trail showing the difficulty progression of answered questions (Beginner, Intermediate, Advanced). After answering each question, show instant AI evaluation feedback, a dynamic difficulty shift banner indicating whether the next question was escalated or de-escalated, and complete the session with an interactive score summary and canvas confetti animation.

4. Backend & Document Ingestion Prompts

PROMPT 3.1 — 300MB MULTER ASYNCHRONOUS INGESTION

Configure an asynchronous file upload pipeline in Express using Multer. Set the upload limit to 300MB to support dense, multi-hundred-page textbooks and scanned presentation decks. Handle Multer exceptions cleanly: if LIMIT_FILE_SIZE is triggered, return a structured 400 Bad Request JSON instead of crashing the server. Offload uploaded file paths to an asynchronous background worker queue (jobQueue.js) for layout parsing and concept discovery.

PROMPT 3.2 — PDF COORDINATE-PRESERVING LAYOUT PARSER

Integrate pdf-parse to extract text from multi-page documents while preserving reading order and layout coordinates. Extract text page by page, demarcating page breaks with explicit page numbers. Strip scanner watermarks like 'Scanned by CamScanner' or 'Adobe Scan'. If extracted page text density is below 50 alphanumeric characters per page, mark the document as scanned and trigger the Multimodal Vision OCR pipeline.

5. Database Design & Batch Chunking Prompts

PROMPT 4.1 — 400-CHUNK BATCH INGESTION

Design the MaterialChunk Mongoose schema to store chunkIndex, pageNumber, materialId, projectId, content, keywords, and tokenCount. When processing large 100 to 500+ page documents, batch insert chunks in sets of 400 documents using MaterialChunk.insertMany(batch, { ordered: false }) to prevent MongoDB 16MB BSON limits and network timeouts. Create compound indexes on { projectId: 1, pageNumber: 1 } and { projectId: 1, chunkIndex: 1 }.

PROMPT 4.2 — CONCEPT KNOWLEDGE GRAPH MODEL

Create a Concept model linked to projects. Each concept node must track title, description, prerequisites (array of concept titles), mastery level (0 to 100%), and lastAssessed timestamp. Write an upsert routine that merges extracted concepts without creating duplicates, decaying or boosting mastery levels based on student quiz attempts.

6. AI & Multimodal OCR Engineering Prompts

PROMPT 5.1 — GEMINI MULTIMODAL VISION OCR FOR HANDWRITTEN NOTES

You are an expert OCR transcription engine for handwritten notes, study materials, and technical documents. Transcribe the handwritten notes and content from Page `${startPage}` to Page `${endPage}` verbatim.

CRITICAL INSTRUCTIONS:

1. Clearly demarcate each page with the header: `=== Page X ===` (e.g. `=== Page ${startPage} ===`).
2. Transcribe all text, handwritten notes, SQL queries, code snippets, definitions, bullet points, numbered steps, headings, and formulas exactly as written.
3. If diagrams or flowcharts are drawn, transcribe all labels, node titles, and flow arrows cleanly.
4. Do NOT summarize or invent facts. Transcribe strictly what is present on the pages.

PROMPT 5.2 — GROUNDED AI TUTOR SYSTEM PROMPT

You are the AI Study Companion Tutor for this learning workspace.

Project Title: `"${projectTitle}"`

Target Learning Goal: `"${learningGoal}"`.

CRITICAL TUTORING RULES:

1. STRICT GROUNDING IN THE PROJECT: Rely strictly on facts, findings, chapters, data, commands, syntax, and concepts from the uploaded documents.
2. CLEAN FORMATTING & NO ASTERISKS: DO NOT USE DOUBLE ASTERISKS (`**`) FOR BOLDING. NEVER write `**word**`. Use clean bullet points (`•`) and clear markdown headings (`### Header`) without asterisks.
3. STRICT RESTRICTION TO UPLOADED MATERIALS: You are STRICTLY RESTRICTED to answering based solely on the provided Evidence Context. If the user asks about general knowledge, historical figures, politics, or topics not in the notes (e.g. 'father of nation'), DO NOT answer using outside knowledge. Explicitly state that the topic is not covered in the uploaded materials.

PROMPT 5.3 — DYNAMIC COMPUTER-ADAPTIVE QUIZ QUESTION GENERATION

Generate a single adaptive quiz question tailored to the student's current proficiency.

Target Difficulty: `"${targetDifficulty}"` (options: beginner, intermediate, advanced).

Previous Question Context: `"${previousFeedback}"`.

Target Concepts: `${conceptList}`.

Return a strict JSON object with question, type (`'mcq'` or `'open_ended'`), options (for mcq), correctAnswer (index for mcq), explanation, rubric, and targetedConcept.

PROMPT 5.4 — OPEN-ENDED RUBRIC GRADING PROMPT

You are an expert pedagogical grader. Evaluate the student's answer to the following conceptual question:

Question: `"${question.question}"`

Rubric: `"${question.rubric}"`

Student Answer: `"${userAnswer}"`

Return a JSON object containing: score (0 to 100), understandingSummary (bulleted breakdown of key insights demonstrated), missingKeyElements (what was omitted), feedback (constructive advice), and conceptDelta (+5 to +15 for mastery, -5 to -15 for misconceptions).

7. Debugging & Root-Cause Elimination Prompts

PROMPT 6.1 — GEMINI RATE LIMIT (HTTP 429) AUTOMATIC FAILOVER

When using the primary Gemini 3.5 Flash model, free-tier keys frequently encounter HTTP 429 quota exhaustion or HTTP 503 high-demand errors. Refactor `callGemini` in `aiService.js` to maintain a candidate fallback list: `['gemini-3.5-flash', 'gemini-3.1-flash-lite', 'gemini-3.5-flash-lite', 'gemini-2.5-flash']`. If the active model returns 429, 404, or 503, automatically rotate to the next candidate model seamlessly. If all API models fail, fall back to the in-house deterministic Content Engine.

PROMPT 6.2 — HEADING TOKEN SUBSTRING MATCH BUG FIX

In `retrievalScoring.js`, heading boost regex currently uses `new RegExp('[A-Za-z0-9\\s,&--/-]*' + tok + '[A-Za-z0-9\\s,&--/-]*')`. When `tok` is `'nation'`, this regex matches any word containing `'nation'` as a substring, including `'combination'`, `'explanation'`, and `'elimination'`. Fix this by adding strict word boundaries: `\\b${tok}\\b` so that heading boosts only trigger on exact whole words.

8. Testing & Performance Benchmarking Prompts

PROMPT 7.1 — 500-PAGE TEXTBOOK RAG BENCHMARK

Write a standalone Node.js benchmark (test-500-pages-rag.js) that generates 500 synthetic technical textbook pages with 560,000+ characters, chapter headings, and distinct index terms. Measure:

1. Chunking latency (must complete under 50ms).
2. Uniform representative sampling across 500 pages (must stay under 30,000 characters).
3. Retrieval latency across 1,000 in-memory chunks for exact page intent, page ranges, and overview queries (must execute in under 60ms).

Assert that all latency thresholds pass.

PROMPT 7.2 — END-TO-END AUTOMATED REGRESSION SUITE

Create a complete automated test runner (backend/tests/run-tests.js) testing all core API endpoints:

1. User authentication (JWT login)
2. Admin authentication (RBAC verification)
3. Space and project listing
4. Grounded AI Tutor queries with verified page citations
5. Out-of-domain query rejection (sourdough recipe, father of nation)
6. Adaptive quiz generation with MCQ and Open-ended questions
7. Quiz submission and rubric grading
8. Concept mastery updates
9. Admin observability telemetry

Ensure all 9 tests pass with 100% success rate.

9. Documentation & Production Deployment Prompts

PROMPT 8.1 — PRODUCTION STARTUP BOOTSTRAP SEEDER

When deploying the backend to Render, the database starts empty or lacks default administrator credentials. Create a bootstrap routine (bootstrap.js) that is called automatically on server startup after connectDB(). Ensure that admin@gmail.com (password: admin123) and bindu@gmail.com (password: password123) are verified or created idempotently on every boot, guaranteeing zero login failures upon fresh deployment.

PROMPT 8.2 — PRODUCTION BUILD & DEPLOYMENT HARDENING

Prepare the repository for dual deployment on Netlify (Frontend) and Render (Backend). Create netlify.toml configuring SPA rewrites (/* to /index.html). Configure render.yaml for the Node.js backend. Run frontend build verification (vite build) ensuring zero errors and clean chunk bundling.